

Using Monte Carlo in conjunction with Euler Discretization in Python

In this chapter, we will be looking at some more advanced features of finance and mathematics. We will use the Euler Discretization technique to calculate the call option. This is a more sophisticated way of calculating the call option than the techniques we discussed in the previous section. To compare the two approaches, we will use the same dataset we used in the previous section, which is the Procter and Gamble dataset for the time period from January 1, 2007 until March 21, 2017.

To begin with, let's import all the necessary libraries, as shown here:

```
In[1]:
import numpy as np
import pandas as pd
from pandas_datareader import data as web
from scipy.stats import norm
import matplotlib.pyplot as plt
%matplotlib inline
```

The next step is to read the dataset using the pandas `read_csv()` method, as shown here:

```
In[2]:
data = pd.read_csv('PG_2007_2017.csv', index_col = 'Date')
```

In the next step, we calculate the log returns, as shown here:

```
In[3]:
log_returns = np.log(1 + data.pct_change())
```

In Euler Discretization, the methods and formula we will apply to compute the call option price are different. We want to run a huge number of experiments to make sure that the price we pick is the most accurate one.

Monte Carlo simulation can provide us with thousands of possible call option prices. We can then average the pay off. The trick lies in the formula we will use to calculate the future prices, which is as follows:

$$S_t = S_{t-1} \cdot e^{((r - \frac{1}{2} \cdot \text{stdev}^2) \cdot \delta_t + \text{stdev} \cdot \sqrt{\delta_t} \cdot Z_t)}$$

This is another version of a Brownian motion. The formula and the approach employed are what is known as Euler Discretization. The following are the parameters of the preceding equation:

- S_t = the stock price for day t
- S_{t-1} = the stock price observed on the previous day, $t-1$

Let's go through the steps that will allow us to assign values in this long formula.

The first thing that we will do is assign a risk-free interest rate (r). The initialization is shown here:

```
| r = 0.025
```

Here, we assign the value of the risk-free interest rate as 2.5%. Next, we can obtain the standard deviation of the log returns and store it in a `stdev` variable, as shown here:

```
| In[5]:  
| stdev = log_returns.std() * 250 ** 0.5  
| stdev
```

The output is as follows:

```
| Out[5]:  
| PG      0.176109  
| dtype: float64
```

We can verify that the `stdev` variable is a series using the following code:

```
| In[6]:  
| type(stdev)
```

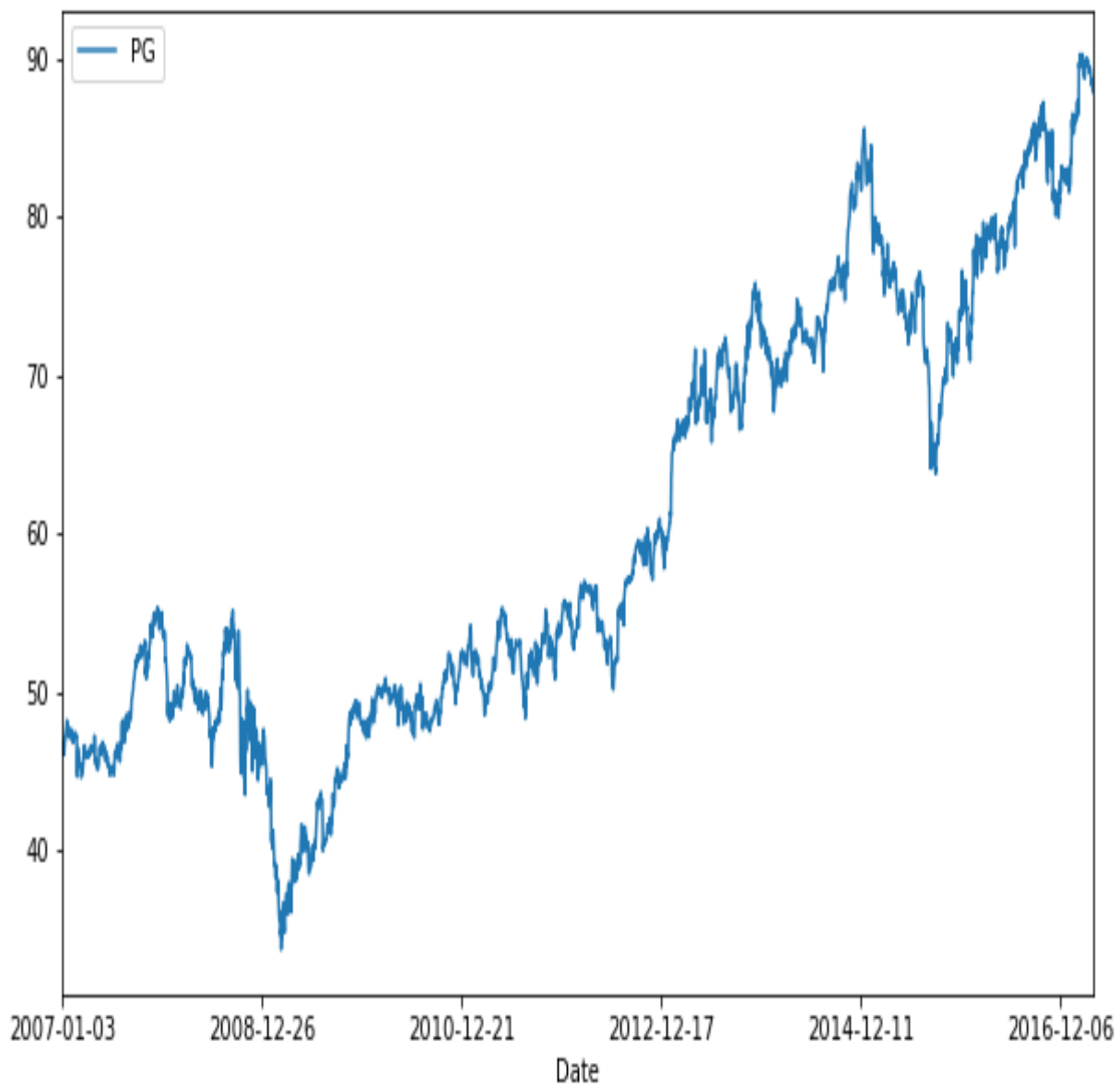
The output is as follows:

```
Out[6]:  
pandas.core.series.Series
```

Let's plot and check the data of the Proctor and Gamble dataset, as shown here:

```
In[6]:  
data.plot(figsize=(10, 6));
```

The output is as follows:



We intend to use the `stdev` variable in an array operation afterwards, so it is necessary to convert the `stdev` variable, which is a series, into an array, by using `values`, as shown here:

```
In[7]:
stdev = stdev.values
stdev
```

The output is as follows:

```
Out[7]:
array([ 0.17610875])
```

The preceding output basically tells us that `stdev` is now a NumPy array.

We will consider the time (τ) to be 1 year, since we are forecasting the prices for 1 year ahead. The number of time intervals (`t_intervals`) must correspond to the number of trading days in a year, which is 250. The initialization is shown in the following code:

```
In[8]:
T = 1.0
t_intervals = 250
delta_t = T / t_intervals

iterations = 10000
```

From the preceding code, we have created two more variables: `delta_t`, which is the fixed time interval, and the 10,000 simulations or iterations for simulating z , which is the random component.

The code is shown here:

```
In[9]:
Z = np.random.standard_normal((t_intervals + 1, iterations))
```

The random component z will be a matrix with random components drawn from a standard normal distribution, which is a normal distribution with a mean of 0 and a standard deviation of 1.

The dimension of the matrix will be defined by the number of time intervals augmented by 1 (`t_intervals + 1`) and the number of iterations (`iterations`).

The code is as follows:

```
In[10]:
S = np.zeros_like(Z)
```

In this step, we can create an empty array, `s`, of the same dimensions as the random component `z`. For this, we use the `np.zeros_like()` method, as shown in the preceding code.

Then, we will use the `iloc` operator with the argument as `-1` to get the last record. We will store the last record in the variable `s0`, as shown in the preceding code:

```
In[11]:
S0 = data.iloc[-1]
S[0] = S0
```

As well as changing the formula, which differs from the one we used for calculating the future stock prices, the remaining steps are almost identical to the Monte Carlo technique we looked at in the previous chapter.

We loop from `1` to the number of intervals (`t_intervals + 1`) and we create a whole stock price matrix with the dimensions of (`t_intervals + 1`) and the number of iterations, as shown here:

```
In[12]:
for t in range(1, t_intervals + 1):
    S[t] = S[t-1] * np.exp((r - 0.5 * stdev ** 2) * delta_t + stdev * delta_t **
0.5 * Z[t])
```

We can see the value inside `s`, as follows:

```
In[13]:
S
```

The output is as follows:

```
Out[13]:
array([[ 88.118629 ,  88.118629 ,  88.118629 , ...,  88.118629 ,
        88.118629 ,  88.118629 ],
       [ 88.81636639,  90.61639177,  89.03898033, ...,  87.30375254,
        87.5583302 ,  88.80166136],
       [ 86.90009657,  91.62555653,  89.63268689, ...,  88.33679858,
        87.62299858,  91.35123942],
       ...,
       [ 84.3786526 , 107.72224183,  80.82567392, ..., 125.62838195,
```

```
|      89.32732597, 105.18086595],
|      [ 84.3872724 , 108.28857142, 81.12333622, ..., 125.84132277,
|      90.90533155, 105.51622291],
|      [ 83.67055401, 107.96691607, 81.7082641 , ..., 125.94172348,
|      91.80990436, 104.04308705]])
```

We can check the dimension by using the `shape` property, as shown here:

```
|In[14]:
|S.shape
```

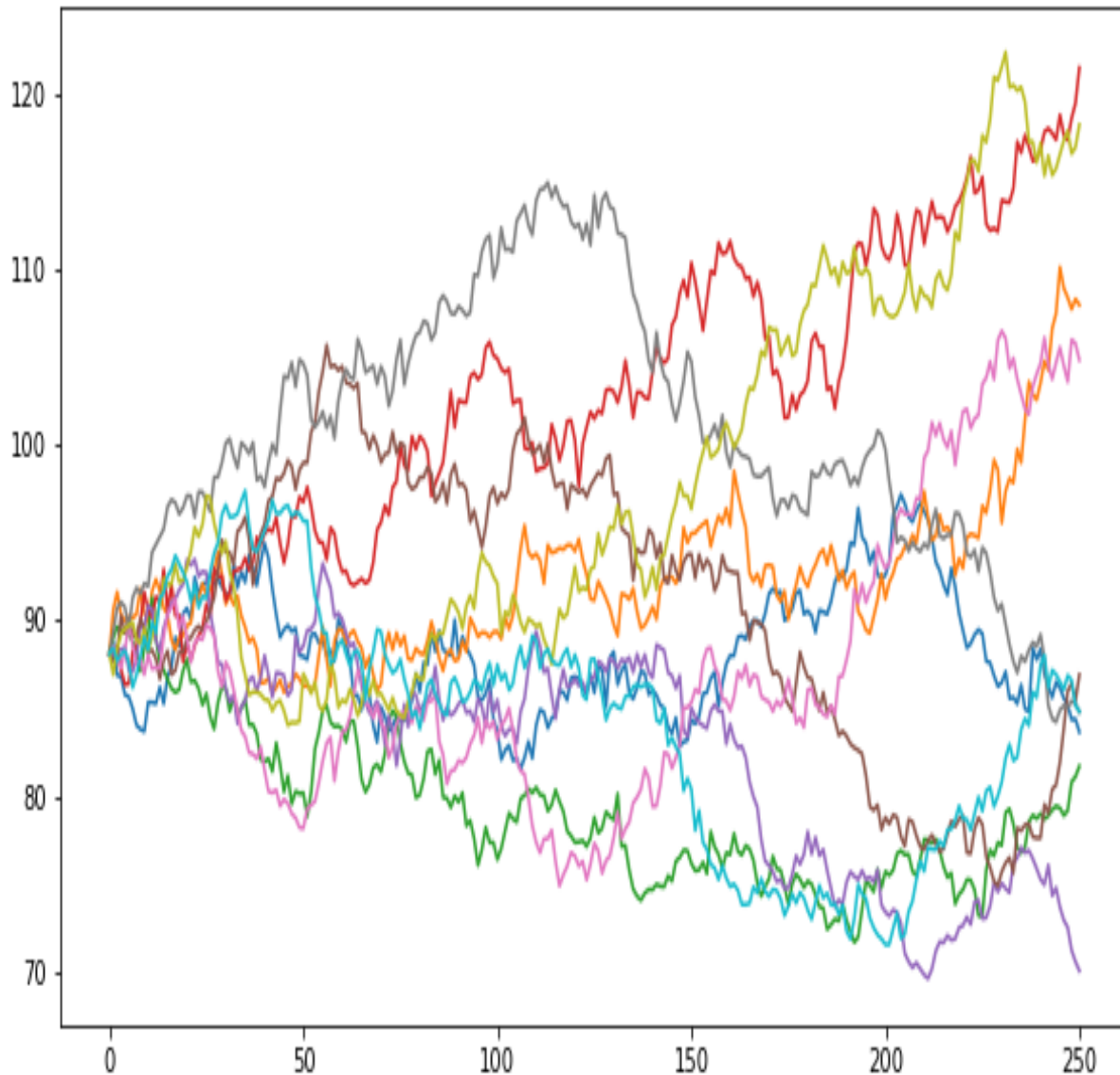
The output is as follows:

```
|Out[14]:
|(251L, 10000L)
```

To plot only 10 simulations, we can use `matplotlib`. The code is shown here:

```
|In[15]:
|plt.figure(figsize=(10, 6))
|plt.plot(S[:, :10]);
```

The output is as follows:



Our simulation is complete and now we will go ahead and compute the call option price.

Let's see what the payoff is like for a call option. At a certain point in time, we will exercise our right to buy the option if the stock price minus the strike price is greater than zero. We will not exercise our right to buy if the difference is a negative number.

So, the value of the option depends on the chance of the difference between the stock price (S) and the strike price (K) being positive and, in particular,

how positive it is expected to be.

To work this out, we can use a NumPy method called `maximum()`, which will create an array that contains either zeros or the numbers equal to the differences. We call `p`, which represents the payoff. The code is as follows:

```
In[16]:  
p = np.maximum(S[-1] - 110, 0)  
p
```

The output is as follows:

```
Out[16]:  
array([ 0.,  0.,  0., ...,  0.,  0.,  0.])
```

We can also check the shape of `p`, as shown here:

```
In[17]:  
p.shape
```

The dimensions are as follows:

```
Out[17]:  
(10000L,)
```

The formula to discount the average of the payoff p is shown here:

$$C = \frac{e^{(-rT)} \cdot \sum p_i}{iterations}$$

This corresponds to the exponential of the r and T multiplied by the sum of the computed payoffs, divided by the number of iterations we choose, which is 10,000.

The preceding equation can be written in Python, as shown here:

```
In[18]:  
C = np.exp(-r * T) * np.sum(p) / iterations  
C
```

The output is the price of the call option:

```
Out[18]:  
1.159769289926591
```


It is important to compare whether this number differs substantially from the one we obtained with the Black Scholes formula in the previous section. From the Black Scholes formula, we got a call option value of 1.13. This is very close to the value of 1.15, which we got by using Euler Discretization. This is proof that the choice of computation method can lead to insignificant but not unimportant differences in the outcome.

We are now armed with enough pythonic skills to conduct these kinds of studies on our own.